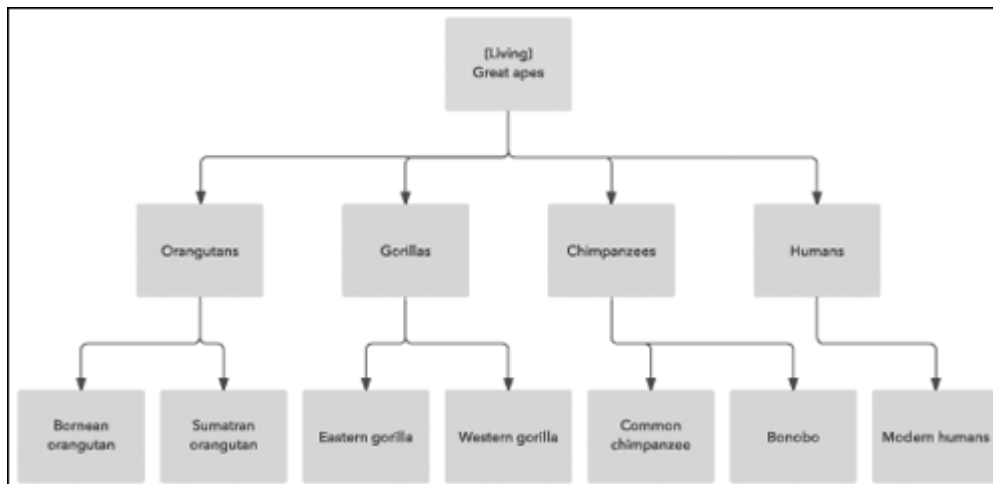


09. Les modèles de données

Bases de données SQL et NoSQL

cf. Annexe 02

07a. Modèle Hiérarchique



Présentation

Le modèle de données hiérarchique organise les données en une structure en arbre où chaque enregistrement a un seul parent, mais peut avoir plusieurs enfants. Initialement utilisé dans les premiers systèmes de gestion de bases de données, ce modèle est simple et permet une navigation rapide à travers les structures.

Utilité dans le Développement Logiciel

Organisation des Données :

Les données sont structurées de manière hiérarchique, ce qui est particulièrement utile pour représenter des informations organisées naturellement sous cette forme, comme les structures d'entreprise, les catégories de produits, ou les dossiers et sous-dossiers sur un système de fichiers.

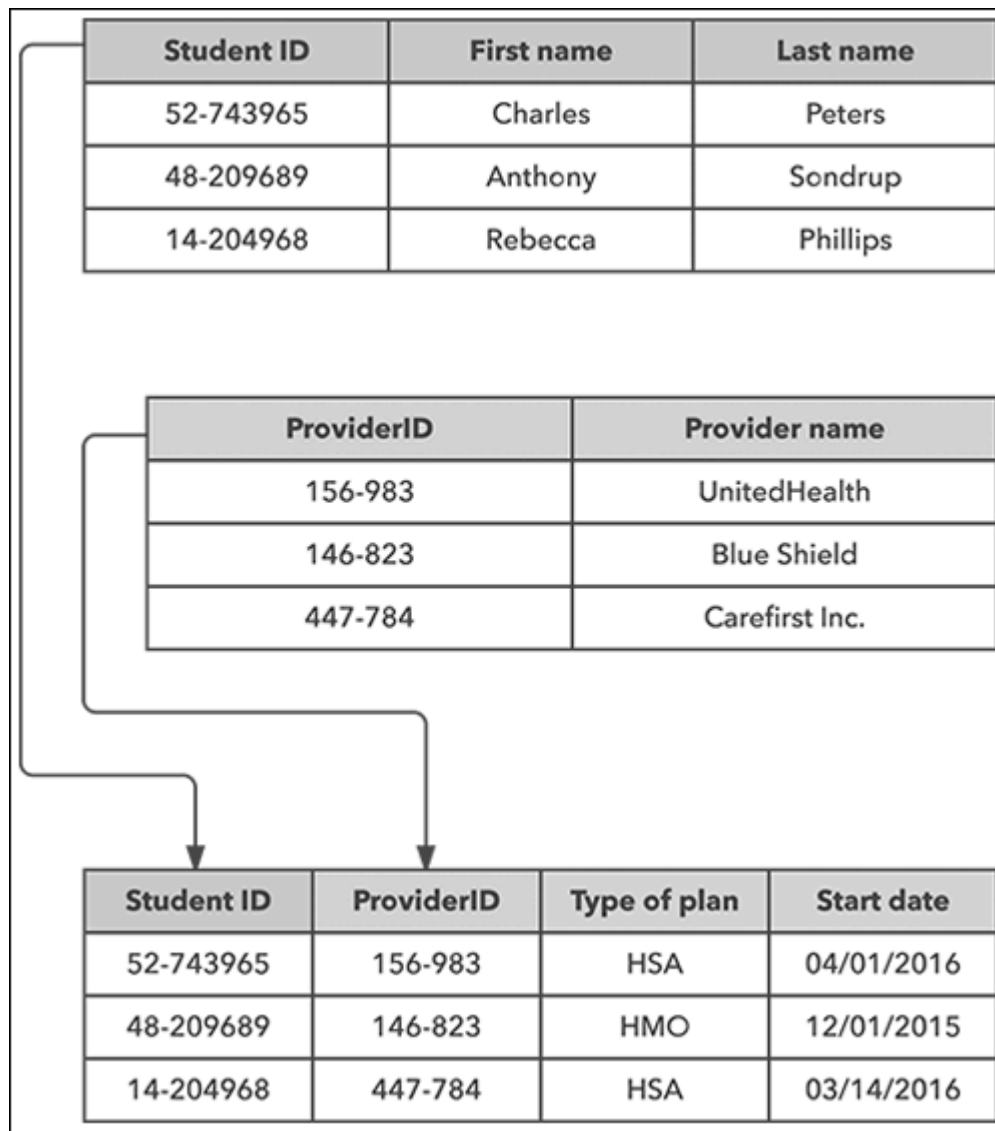
Requêtes Rapides :

La navigation et les requêtes peuvent être très rapides si les relations parent-enfant sont clairement définies, car le chemin d'accès à chaque noeud est unique.

Symboles

Les symboles utilisés dans le modèle hiérarchique incluent principalement les noeuds et les liens qui les connectent. Chaque noeud représente un enregistrement ou un objet, et les liens représentent les relations de parenté à enfant.

09b. Modèle Relationnel



Présentation

Le modèle relationnel, proposé par Edgar F. Codd, organise les données en tables (ou relations) composées de lignes et de colonnes. Chaque table représente un type d'entité, et chaque ligne (ou tuple) une instance de cette entité. Les relations entre les tables sont gérées par des clés étrangères.

Utilité dans le Développement Logiciel

Flexibilité des Requêtes :

Grâce à l'algèbre relationnelle et au SQL, ce modèle offre une grande flexibilité pour écrire des requêtes complexes et réaliser des jointures entre tables.

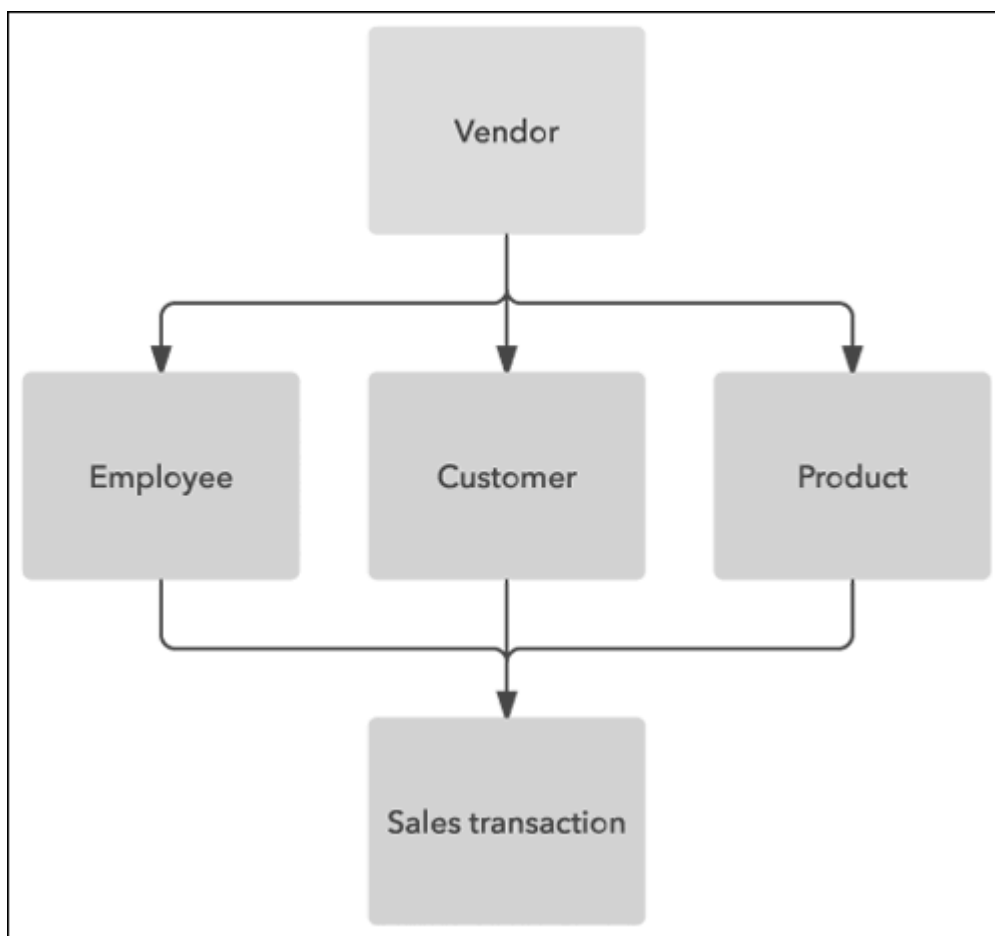
Intégrité des Données :

Le modèle relationnel permet une gestion efficace de l'intégrité des données à travers des contraintes de clés primaires et étrangères.

Symboles

Les symboles principaux incluent les rectangles représentant les tables, et les lignes qui relient ces tables pour indiquer les relations. Les clés primaires et étrangères sont souvent mises en évidence dans les schémas.

09c. Modèle Réseau



Présentation

Le modèle de données réseau est une extension du modèle hiérarchique où chaque enregistrement peut avoir plusieurs parents. Ce modèle permet une plus grande flexibilité par rapport au modèle strictement hiérarchique.

Utilité dans le Développement Logiciel

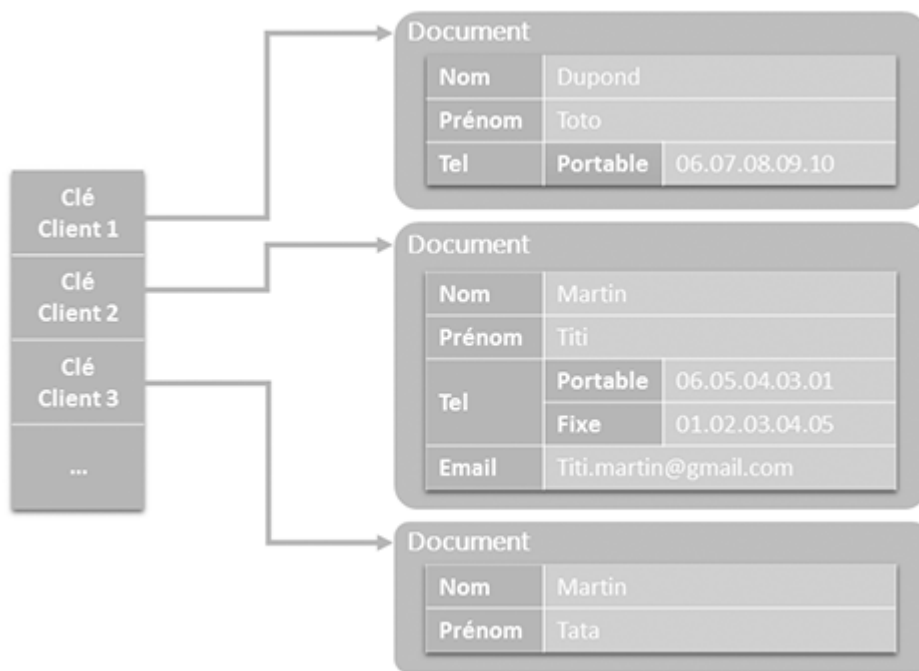
Flexibilité des Relations :

Les relations complexes entre enregistrements peuvent être plus naturellement modélisées, ce qui est utile pour des applications nécessitant de multiples relations entre les données, comme les systèmes de réservation ou les réseaux sociaux.

Symboles

Comme pour le modèle hiérarchique, les symboles comprennent des noeuds et des connexions, mais avec la possibilité de multiples connexions entrantes pour chaque noeud, reflétant la structure de réseau.

09d. Modèle Document



Présentation

Le modèle de données basé sur des documents stocke les informations sous forme de documents (souvent formatés en JSON ou XML). Chaque document peut contenir une structure de données complexe avec des champs imbriqués.

Utilité dans le Développement Logiciel

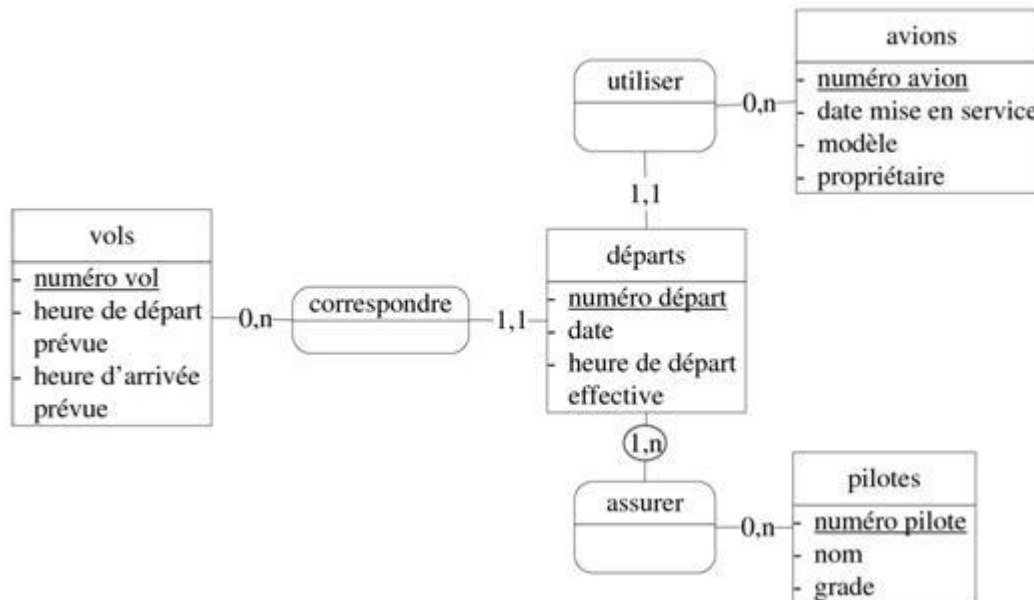
Souplesse de Schéma :

Le modèle document est très flexible, permettant des modifications de schéma sans interruption de service, idéal pour les applications nécessitant une évolution rapide et des structures de données variées.

Symboles

Les documents sont souvent représentés par des icônes ressemblant à des documents ou des fichiers, avec des liaisons entre eux pour montrer les relations imbriquées ou les références.

09e. Modèle Entité / Association



Présentation

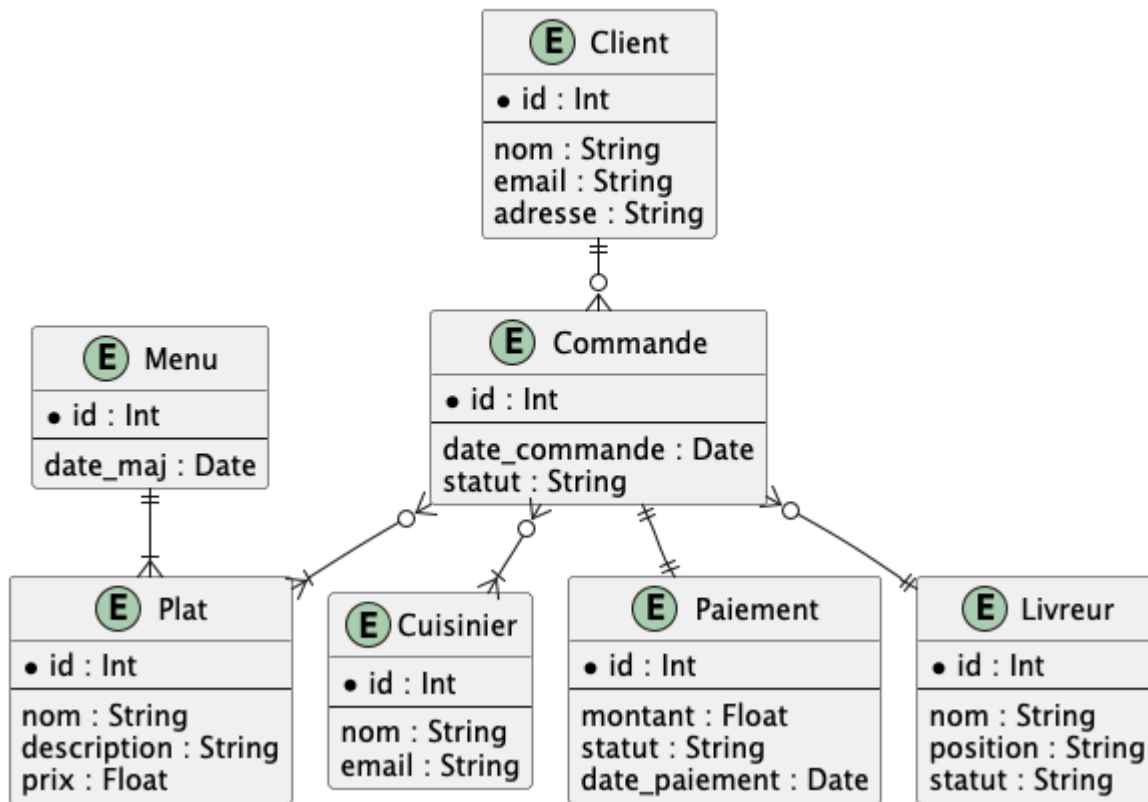
- **Origine** : Le modèle EA, souvent utilisé dans la méthode MERISE, a été développé en France dans les années 1970. MERISE est une méthode d'analyse et de conception des systèmes d'information qui utilise le modèle EA pour représenter les données.
- **Objectif** : Le modèle EA vise à fournir une représentation claire et simple des données et des relations dans un système d'information, en se concentrant sur les entités et leurs associations.

Symboles

Les entités sont représentées par des rectangles, et les associations par des lignes les reliant, souvent avec des labels pour décrire la nature de la relation. Les attributs peuvent être listés dans ou autour des rectangles d'entité. Ce modèle utilise des bulles pour représenter les associations entre entités. Les cardinalités sont souvent notées sous la forme "1,n".

09f. Modèle Entité / Relation

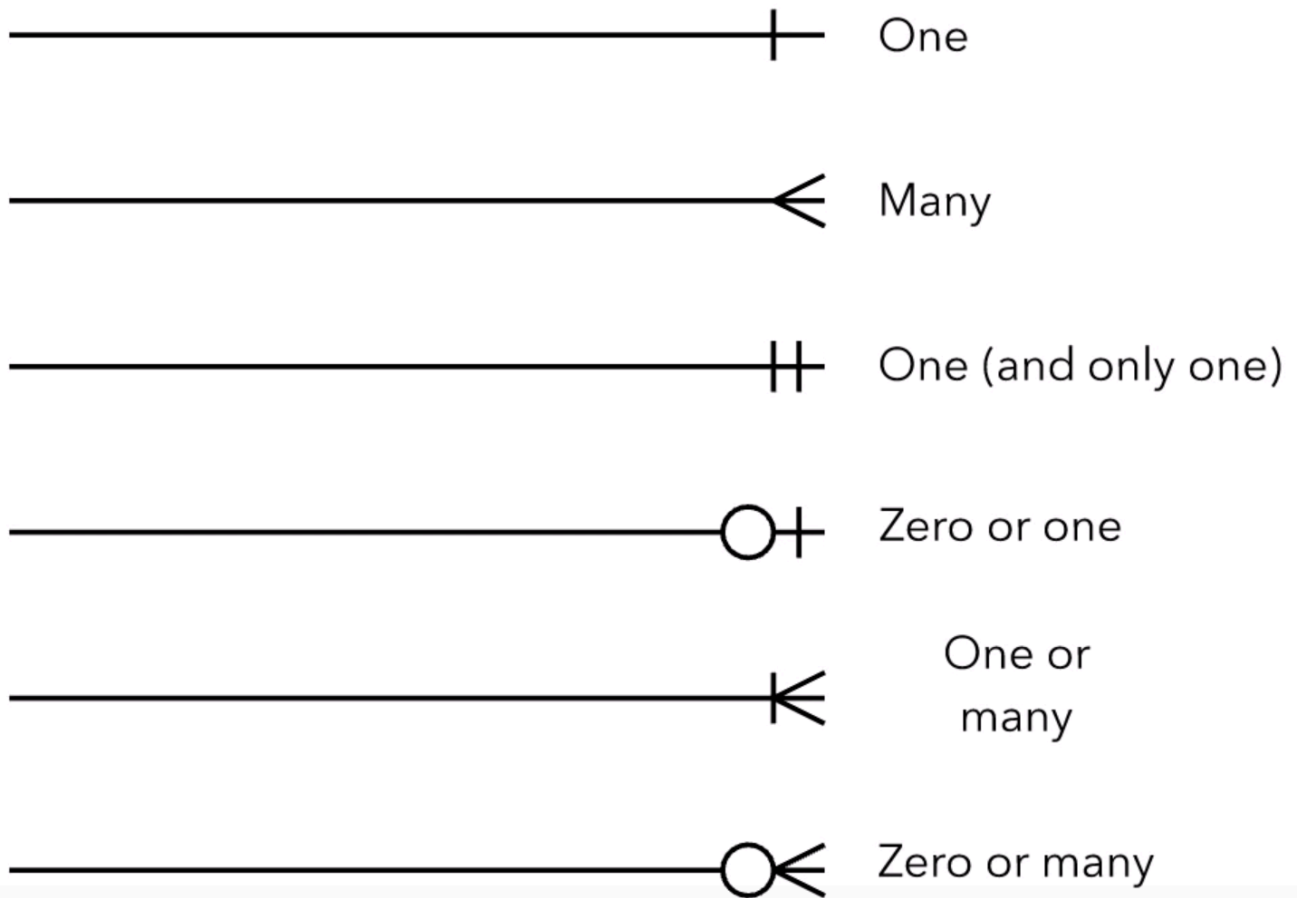
Entité-Relation : Gestion des commandes de repas en ligne



Présentation

- **Origine** : Le modèle ER a été introduit par Peter Chen en 1976 dans son article "The Entity-Relationship Model - Toward a Unified View of Data". Ce modèle est devenu largement adopté dans le monde entier pour la modélisation des bases de données relationnelles.
- **Objectif** : Le modèle ER vise à fournir une représentation graphique plus structurée et formelle des entités, des attributs et des relations dans une base de données.

Symboles



Le modèle ER utilise des traits verticaux, des pattes d'oie, et des ronds pour indiquer les cardinalités des relations entre les entités.

Les modèles suivants sont des dérivés, ils n'appartiennent pas à UML

07g. Modèle Clé-valeur

Key-Value Store	
Key	Value
user_123	{ "name": "John Doe", "email": "john.doe@example.com", "age": 30 }
order_001	{ "orderId": "001", "product": "Laptop", "quantity": 2, "price": 1500 }
order_002	{ "orderId": "002", "product": "Smartphone", "quantity": 1, "price": 800 }

Une base de données clé-valeur est un type de base de données NoSQL où chaque donnée est stockée sous la forme d'une paire clé-valeur. Ce modèle est extrêmement simple mais très puissant pour des opérations rapides de lecture/écriture.

Description du Schéma

- **Clé (Key)** : Une clé unique identifie chaque paire clé-valeur. La clé peut être un identifiant utilisateur (user_123), un identifiant de commande (order_001), etc.
- **Valeur (Value)** : La valeur associée à la clé peut être une chaîne de caractères, un nombre, un tableau, ou un objet (souvent au format JSON). La valeur contient les données réelles.

Exemples de Cas d'Utilisation

- **Stockage de Sessions Utilisateur** : Enregistrer les informations de session des utilisateurs dans une application web.
- **Cache de Données** : Stocker des résultats de requêtes pour une récupération rapide.
- **Stockage de Configuration** : Enregistrer les paramètres de configuration d'une application.

Avantages

- **Simplicité** : Modèle de données simple et facile à comprendre.
- **Performance** : Excellente performance pour les opérations de lecture/écriture rapides.
- **Flexibilité** : Aucune contrainte de schéma, permettant une grande flexibilité dans le stockage des données.

Limites

- **Requêtes Complexes** : Pas conçu pour des requêtes complexes ou des relations entre les données.
- **Évolutivité des Données** : Peut devenir difficile à gérer si les données deviennent trop complexes et interreliées.

07h. Modèle en Colonne

Column Family: Users				
Row Key	Name	Email	Age	
user_123	John Doe	john.doe@example.com	30	
user_124	Jane Smith	jane.smith@example.com	25	

Column Family: Orders				
Row Key	Product	Quantity	Price	Date
order_001	Laptop	2	1500	2023-01-01
order_002	Smartphone	1	800	2023-01-02

Une base de données orientée colonne est un type de base de données NoSQL où les données sont stockées par colonnes plutôt que par lignes. Cela permet une lecture rapide et efficace des données lorsque des opérations analytiques sur un grand nombre de lignes mais un petit nombre de colonnes sont requises.

Description du Schéma

- **Column Family** : Une collection logique de colonnes, similaire à une table dans une base de données relationnelle.
 - **Users** : Une column family pour stocker les informations des utilisateurs.
 - **Orders** : Une column family pour stocker les informations des commandes.
- **Row Key** : Une clé unique qui identifie chaque ligne au sein d'une column family.

- **Exemples** : user_123, user_124 pour les utilisateurs; order_001, order_002 pour les commandes.
- **Colonnes** : Chaque column family contient des colonnes qui stockent les valeurs des données.
 - **Users** : Colonnes Name, Email, Age.
 - **Orders** : Colonnes Product, Quantity, Price, Date.

Exemples de Cas d'Utilisation

- **Analyse de Données Massives** : Optimisé pour les requêtes analytiques sur de grands volumes de données.
- **Systèmes de Recommandation** : Stockage et accès rapides aux données comportementales des utilisateurs.
- **Entreposage de Données** : Idéal pour les entrepôts de données et les systèmes OLAP (Online Analytical Processing).

Avantages

- **Efficacité de Lecture** : Très efficace pour les opérations de lecture sur des colonnes spécifiques, particulièrement avantageux pour les requêtes de type analytique.
- **Compression** : Meilleure compression des données car les colonnes similaires sont stockées ensemble.
- **Scalabilité Horizontale** : Conçu pour être facilement évolutif horizontalement en ajoutant plus de nœuds.

Limites

- **Complexité des Requêtes** : Moins adapté pour les requêtes complexes qui nécessitent des jointures entre plusieurs tables ou column families.
- **Modélisation des Données** : Peut nécessiter une planification minutieuse de la modélisation des données pour maximiser les avantages.

091. Modèle en Graphe

```

[NODE: User]
+-----+
| Node ID | Properties |
+-----+
| user_123 | {"name": "John Doe", "age": 30} |
| user_124 | {"name": "Jane Smith", "age": 25} |
+-----+

[NODE: Product]
+-----+
| Node ID | Properties |
+-----+
| product_001 | {"name": "Laptop", "price": 1500} |
| product_002 | {"name": "Smartphone", "price": 800} |
+-----+

[RELATION: PURCHASED]
+-----+
| Relationship ID | From | To | Properties |
+-----+
| purchase_001 | user_123 | product_001 | {"date": "2023-01-01", "quantity": 2} |
| purchase_002 | user_124 | product_002 | {"date": "2023-01-02", "quantity": 1} |
+-----+

[RELATION: FRIEND]
+-----+
| Relationship ID | From | To | Properties |
+-----+
| friend_001 | user_123 | user_124 | {"since": "2022-06-15"} |
+-----+

```

Une base de données orientée graphe est conçue pour représenter les relations entre les données sous forme de graphes, avec des nœuds (entités), des arêtes (relations) et des propriétés (attributs des nœuds et des arêtes). Cela permet une navigation et des requêtes rapides sur des relations complexes.

Description du Schéma

- **Nœuds (Nodes)** : Les entités principales du graphe, telles que les utilisateurs et les produits.
 - **User Nodes** : Représentent des utilisateurs avec des propriétés comme le nom et l'âge.
 - **Product Nodes** : Représentent des produits avec des propriétés comme le nom et le prix.
- **Arêtes (Edges/Relationships)** : Les relations entre les nœuds, qui peuvent avoir leurs propres propriétés.
 - **Purchased Relationship** : Représente une relation d'achat entre un utilisateur et un produit, avec des propriétés comme la date et la quantité.

- **Friend Relationship** : Représente une relation d'amitié entre deux utilisateurs, avec une propriété indiquant la date de début de l'amitié.
- **Propriétés (Properties)** : Attributs des nœuds et des relations, stockés sous forme de paires clé-valeur.

Exemple de Cas d'Utilisation

- **Réseaux Sociaux** : Modéliser les utilisateurs et leurs relations (amis, abonnements, etc.).
- **Recommandations de Produits** : Basé sur les achats et les relations entre les utilisateurs.
- **Détection de Fraude** : Identifier des motifs de fraude à travers les relations complexes entre les entités.

Avantages

- **Navigation Rapide des Relations** : Permet de traverser rapidement les relations complexes entre les données.
- **Flexibilité des Schémas** : Facile à ajouter de nouveaux types de nœuds et de relations sans restructurer la base de données.
- **Analyses Complexes** : Parfait pour des analyses complexes comme les graphes de connaissances, les réseaux sociaux et les systèmes de recommandation.

Limites

- **Performance** : Peut être moins performant pour les opérations massives de lecture/écriture non relationnelles.
- **Complexité** : Modélisation et requêtes peuvent être plus complexes comparées à d'autres types de bases de données NoSQL.

Exemple Pratique avec Cypher (langage de requête pour Neo4j)

```
// Créer des nœuds utilisateurs
CREATE (u1:User {id: 'user_123', name: 'John Doe', age: 30})
CREATE (u2:User {id: 'user_124', name: 'Jane Smith', age: 25})

// Créer des nœuds produits
CREATE (p1:Product {id: 'product_001', name: 'Laptop', price: 1500})
CREATE (p2:Product {id: 'product_002', name: 'Smartphone', price: 800})

// Créer des relations d'achat
CREATE (u1)-[:PURCHASED {date: '2023-01-01', quantity: 2}]->(p1)
CREATE (u2)-[:PURCHASED {date: '2023-01-02', quantity: 1}]->(p2)

// Créer une relation d'amitié
CREATE (u1)-[:FRIEND {since: '2022-06-15'}]->(u2)
```

Exercices pratiques

Plateforme de Gestion d'Événements

Système de Gestion Universitaire